

W5500 TOE + ESP-IDF lwIP 공존 드라이버 — 소프트웨어 아키텍처

대상: ESP32-S3 + W5500 (단일 칩, 단일 SPI)

목표: 하나의 W5500을 **esp_eth/lwIP(MACRAW)** 와 **ioLibrary/TOE(하드웨어 TCP-IP)** 가 동시에 사용하는 재사용 가능한 드라이버

작성일: 2026-06-01

상태: lwIP 경로 + TOE 경로 양쪽 에코 동작 검증 완료

1. 개요 및 목표

W5500은 8개의 하드웨어 소켓과 16KB(TX)/16KB(RX) 내부 버퍼를 가진 이더넷 컨트롤러다. 본 프로젝트의 목표는 **하나의 W5500 칩을 두 소프트웨어 스택이 공유하게** 하는 드라이버 개발이다.

- esp_eth + lwIP (MACRAW):** 표준 ESP-IDF 네트워크 스택. 풍부한 레퍼런스(BSD socket, DHCP, ICMP, mDNS 등)를 그대로 사용.
- ioLibrary + TOE (TCP/UDP Offload Engine):** W5500 하드웨어가 TCP/IP를 직접 처리. CPU 부하-SPI 트랜잭션이 적어 throughput 유리.

단일 완성품이 아니라 **두 세계를 모두 쓸 수 있게 하는 드라이버**가 핵심 목표다.

2. 하드웨어 구성

2.1 보드

ESP32-S3 + W5500이 **한 보드에 PCB로 고정 배선된** 통합 모듈 (핸드 솔더).

2.2 핀맵 (확정)

W5500 신호	ESP32-S3 GPIO	비고
MISO	GPIO6	
MOSI	GPIO9	
SCLK	GPIO8	
CS (SCSn)	GPIO7	
INT (INTn)	GPIO44	U0RXD
RST (RSTn)	GPIO5	

- SPI host: **SPI2** (`CONFIG_EXAMPLE_ETH_SPI_HOST=1`), GPIO matrix 라우팅.
- 콘솔은 **USB-Serial-JTAG 사용** (UART0 콘솔 비활성). 사유: INT=GPIO44가 UART0(U0RXD) 핀이라, UART0 콘솔과 충돌. 보드의 단일 USB-C가 네이티브 USB(USB-JTAG)이므로 로고는 그쪽으로 출력.

3. 핵심 아키텍처

3.1 소켓 분배

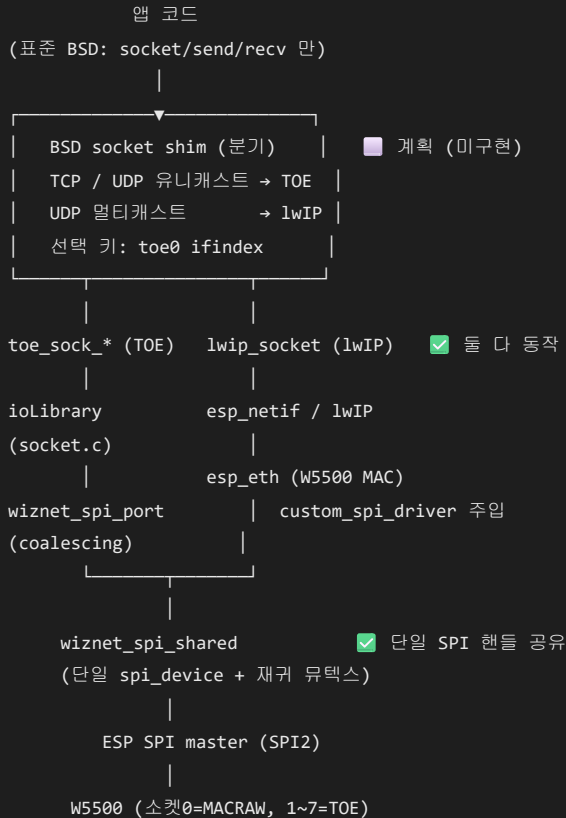
W5500 (소켓 0 ~ 7, 내부 버퍼 16KB → 2KB × 8 분배)

└─ 소켓 0 : MACRAW 모드 → esp_eth → lwIP (L2~L4 소프트웨어 처리)
└─ 소켓 1 ~ 7 : TCP/UDP(TOE) → ioLibrary (L2~L4 하드웨어 처리)

- **소켓 0 (MACRAW):** 들어오는 이더넷 프레임을 그대로 lwIP에 전달. ARP/DHCP/ICMP(ping)/표준 BSD socket이 모두 동작.
- **소켓 1~7 (TOE):** W5500 하드웨어가 핸드셰이크·ACK·재전송까지 직접 처리. CPU는 데이터만 read/write.

3.2 전체 구조도 (목표 아키텍처)

핵심 목표는 앱이 표준 BSD `socket()` / `send()` / `recv()` 만 사용하고, 그 아래 **BSD socket shim(분기 레이어)** 이 경로를 자동으로 가르는 것이다.



레이어	상태
BSD socket shim (분기)	계획 — 다음 단계 (§7.1)
toe_sock_* / lwip_socket (두 경로)	동작 검증
ioLibrary / esp_eth + 공유 SPI 드라이버	
wiznet_spi_shared (단일 핸들)	

현재 상태: shim이 아직 없어서, 앱이 `toe_sock_*`() 와 `lwip_socket()` 을 직접 골라 호출한다(병렬). shim을 없으면 앱은 표준 `socket()` 하나만 쓰고 내부에서 자동 분기된다. (현재 vs 목표 비교는 §7.1)

3.3 단일 SPI 핸들 공유 (핵심 설계)

문제: 칩 하나·CS 하나인데 두 드라이버가 SPI를 두드림. 각자 SPI 디바이스를 만들거나 수동 CS를 토글하면 CS 라우팅이 충돌.

해결: 하나의 `spi_device` 핸들을 `esp_eth`와 `ioLibrary`가 공유.

- `wiznet_spi_shared.c`: `esp_eth`의 W5500 **custom SPI driver**(init/deinit/read/write)를 제공. 단일 `spi_device` 핸들과 재귀 뮉텍스를 소유.
- `ethernet_init.c`: `w5500_config.custom_spi_driver` 에 이 드라이버를 주입 → `esp_eth`가 이 핸들로 SPI 수행.

- `wiznet_spi_port.c` : ioLibrary의 바이트/버스트 콜백을, `esp_eth`와 동일한 프레임 포맷(cmd=16비트 주소, addr=8비트 제어)의 **단일 트랜잭션으로 합쳐서(coalescing)** 같은 핸들로 전송.
- 모든 SPI 접근은 재귀 뮤텍스로 **직렬화**되어 레지스터 깨짐 없이 공존.

3.4 버퍼 분배

`esp_eth` 기본은 소켓0에 16KB 전부, 소켓1~7에 0KB를 할당 → TOE 소켓이 버퍼가 없음. `wiznet_toe_partition_sockets()` 가 **소켓 OPEN 전(esp_eth 시작 전)** 에 8소켓 **2KB씩** 재분배. (제어 트래픽만 타는 MACRAW엔 2KB로 충분, 대용량은 TOE로.)

4. 통합 과정에서 해결한 문제

#	문제	원인	해결
1	칩 리셋 레이스	<code>wizchip_init()</code> 의 <code>MR_RST</code> 가 <code>esp_eth</code> 설정을 통째로 지움	ioLibrary init에서 풀 리셋 호출 제거. 칩 리셋-소켓0 설정은 <code>esp_eth</code> 가 단독 소유
2	버퍼 분배 충돌	<code>esp_eth</code> =소켓0 16KB, 1~7=0KB	소켓 OPEN 전 2KB×8 재분배
3	MACRAW 중복 수신	소켓0 MAC필터가 "내 MAC 유니캐스트"를 다 받음 → TOE 패킷도 소켓0이 받아 lwIP가 RST	부분 미해결 (아래 §7)
4	SPI CS 충돌	두 드라이버가 같은 CS핀을 다른 방식으로 제어	단일 <code>spi_device</code> 핸들 공유 (§3.3)
-	콘솔 UART 충돌	INT=GPIO44=U0RXD, MISO 등과 UART0 핀 겹침	콘솔을 USB-Serial-JTAG로 전환
-	핀맵 오인	거울상 배선도 → MISO/RST 핀 오해	실측 확정 (MISO=6, RST=5)
-	워치독 (디스패처/에코)	<code>vTaskDelay(2ms)</code> 가 100Hz 틱에서 0틱 → busy-spin	<code>CONFIG_FREERTOS_HZ=1000</code> + 딜레이 최소 1틱 보장
-	recv 무한 블로킹	ioLibrary <code>recv()</code> 내부 <code>while(getSn_CR)</code> 등 no-yield busy-wait 가 SPI 경합 시 멈춤	<code>toe_sock_recv</code> 를 직접 구현(데이터 읽기 + RECV + 유한+yield 대기)으로 우회

5. 소프트웨어 구성요소

5.1 신규 컴포넌트: `components/wiznet_toe/`

파일	역할
<code>src/wiznet_spi_shared.c</code>	<code>esp_eth</code> 용 custom SPI 드라이버. 단일 핸들 + 재귀 뮤텍스 소유 (공유의 심장)
<code>src/wiznet_spi_port.c</code>	ioLibrary 콜백 → 공유 핸들 트랜잭션으로 coalescing
<code>src/wiznet_toe.c</code>	TOE 초기화(리셋 없이), 버퍼 분배, 타임아웃
<code>src/toe_netif.c</code>	<code>toe0</code> netif(메타데이터) 생성, eth0 DHCP IP 동기화
<code>src/toe_socket.c</code>	최소 TOE 소켓 API (open/connect/listen/send/ recv 직접구현 /close)
<code>src/toe_dispatcher.c</code>	TOE 소켓 인터럽트 폴링 (현재 에코에션 비활성)
<code>src/toe_demo.c</code>	TOE 에코 풀(소켓 1~7) 데모
<code>lib/ioLibrary_Driver/</code>	WIZnet ioLibrary (socket.c, w5500.c, wizchip_conf.c)

5.2 수정한 기존 컴포넌트

파일	수정
<code>components/ethernet_init/ethernet_init.c</code>	W5500에 custom SPI driver 주입 + 스펙 준수 HW 리셋(10ms low/60ms settle)
<code>main/ethernet_example_main.c</code>	TOE init 순서 조정, <code>toe0</code> 생성/IP 동기화, lwIP 에코 서버(5000), MACRAW 수신 로그 확

파일	수정
sdkconfig	핀맵, 콘솔(USB-JTAG), FREERTOS_HZ=1000, TOE/에코 옵션

5.3 데모 동작 (WIZNET_TOE_ECHO_DEMO=y)

- **lwIP(MACRAW) 에코**: 포트 5000 (표준 BSD socket)
- **TOE 에코 풀**: 소켓 1~7이 각각 포트 5001~5007에서 listen-에코
- **MACRAW 수신 로그**: 소켓0이 받은 프레임마다 출력 (문제3 관찰용)

5.4 초기화 흐름 (app_main)

```

① example_eth_init() → esp_eth 드라이버 + 소켓0 MACRAW + 공유 SPI 드라이버 [main:161]
② wiznet_toe_init() → ioLibrary + 소켓1~7 버퍼분배 (TOE 엔진) [main:169]
③ esp_netif_init() → lwIP 시동 [main:173]
④ eth0 netif 생성 → esp_netif_new + esp_eth_new_netif_glue + attach [main:185]
⑤ esp_eth_start() → MACRAW OPEN (링크업 시) [main:220]
⑥ toe0 netif 생성 → wiznet_toe_netif_create (메타 netif + ifindex) [main:239]

```

"소켓0 = MACRAW" 모드 설정은 우리 코드가 아니라 **esp_eth의 W5500 드라이버 내부**(w5500_setup_default)가 driver_install (①) 시점에 수행한다. 우리는 거기에 **공유 SPI 드라이버만** 주입한다.

항목	MACRAW (eth0)	TOE (toe0)
엔진 init	① example_eth_init	② wiznet_toe_init
netif 생성	④ esp_netif_new + glue + attach	⑥ wiznet_toe_netif_create
lwIP 라우팅	함 (glue로 연결)	안 함 (메타 netif)
시작	⑤ esp_eth_start	(소켓 open 시)

두 경로 모두 같은 W5500 칩 + 같은 SPI 핸들을 공유한다. MACRAW = ①+④+⑤ (esp_eth 표준), TOE = ②+⑥ (본 프로젝트 추가).

각 netif는 독립 esp_netif_t 객체로 관리되어 고유 ifindex를 가진다:

```

int eth_ifindex = esp_netif_get_netif_impl_index(eth_netifs[0]); // MACRAW
int toe_ifindex = esp_netif_get_netif_impl_index(s_toe_netif); // TOE (선택 키)

```

6. 현재 동작 상태 (검증됨)

항목	결과
W5500 칩 인식 (VERSIONR=0x04)	✓
단일 SPI 핸들 공유 (esp_eth + ioLibrary)	✓
eth0 DHCP IP 획득 + toe0 동기화	✓
lwIP/MACRAW : ping, 에코(5000)	✓
TOE : 연결 + 에코(5001~)	✓
워치독 없음 (안정 동작)	✓

→ 단일 W5500을 lwIP와 TOE가 SPI 하나로 공유하며 양쪽 모두 데이터 송수신 동작.

7. 남은 작업

7.1 분기(branch) — 현재는 "수동/병렬"

지금은 자동 분기가 없다. 앱이 직접 API를 선택:

- lwIP 원하면 → 표준 `socket()`
- TOE 원하면 → `toe_sock_*`

두 경로는 독립 병렬로 돌고 SPI 핸들에서만 직렬화된다.

계획(Phase 4, 미구현): 표준 `socket()` 진입점에 **VFS/shim**을 얹어 자동 라우팅

(TCP→TOE, UDP 유니캐스트→TOE, UDP 멀티캐스트→lwIP, 슬롯 부족→lwIP fallback).

7.2 우선순위 과제

1. **toe_sock_send 방어 수정** — `send()` 도 ioLibrary의 SENDOK busy-wait가 남아 있어 부하 시 멈출 수 있음. recv처럼 직접 구현 권장.
2. **문제3 필터** — `macraw_rx_input` 에서 TOE 포트(5001~) 패킷을 lwIP에 올리기 전 drop → lwIP RST 스톱 차단.
3. **자동 분기 shim** — §7.1.
4. **검증 매트릭스** — 7소켓 동시, throughput(iperf), lwIP+TOE 동시 부하, 장시간 안정성.

8. 빌드 / 플래시 노트

- ESP-IDF v5.5.1, 타겟 **esp32s3**.
- venv 파이썬: `idf5.5_py3.11_env` (시스템 `py` 는 3.13이라 export 실패 → 3.11 venv 직접 지정 필요).
- 한글 콘솔(cp949)에서 idf.py 출력 인코딩 깨짐 → `PYTHONUTF8=1` 설정.
- 플래시/모니터 포트: **USB-Serial-JTAG (VID_303A&PID_1001)**. 칩 리부팅 시 COM 번호 재열거릴 수 있음.

부록 A. 핵심 데이터 흐름 (TOE 수신 에코)

```
PC —TCP data—▶ W5500 소켓N(하드웨어 수신/ACK)
|
toe_echo_task 폴링: getSn_RX_RSR > 0
|
toe_sock_recv: wiz_recv_data() —SPI(공유핸들/유크스)—▶ 데이터 read
|   setSn_CR(RECV) + 유한.yield 대기
toe_sock_send: —SPI—▶ TX버퍼 write + SEND
|
W5500 소켓N —TCP data—▶ PC (에코)
```

부록 B. 문제3 상세 (중복수신)

소켓0이 MACRAW + MAC 필터(내 MAC + 브로드캐스트)로 동작 → TOE(소켓1~7)로 가는 유니캐스트 패킷도 소켓0이 중복 수신. `esp_eth`가 이를 lwIP에 올리면, lwIP는 해당 포트 리스너가 없으므로 RST를 보낼 수 있음(실측 확인). 경부하에선 동작하나, 부하 시 RST-SPI 경합으로 성능/안정성 저하 가능 → §7.2-2 필터 필요.